



Aprendiendo a programar con JavaScript

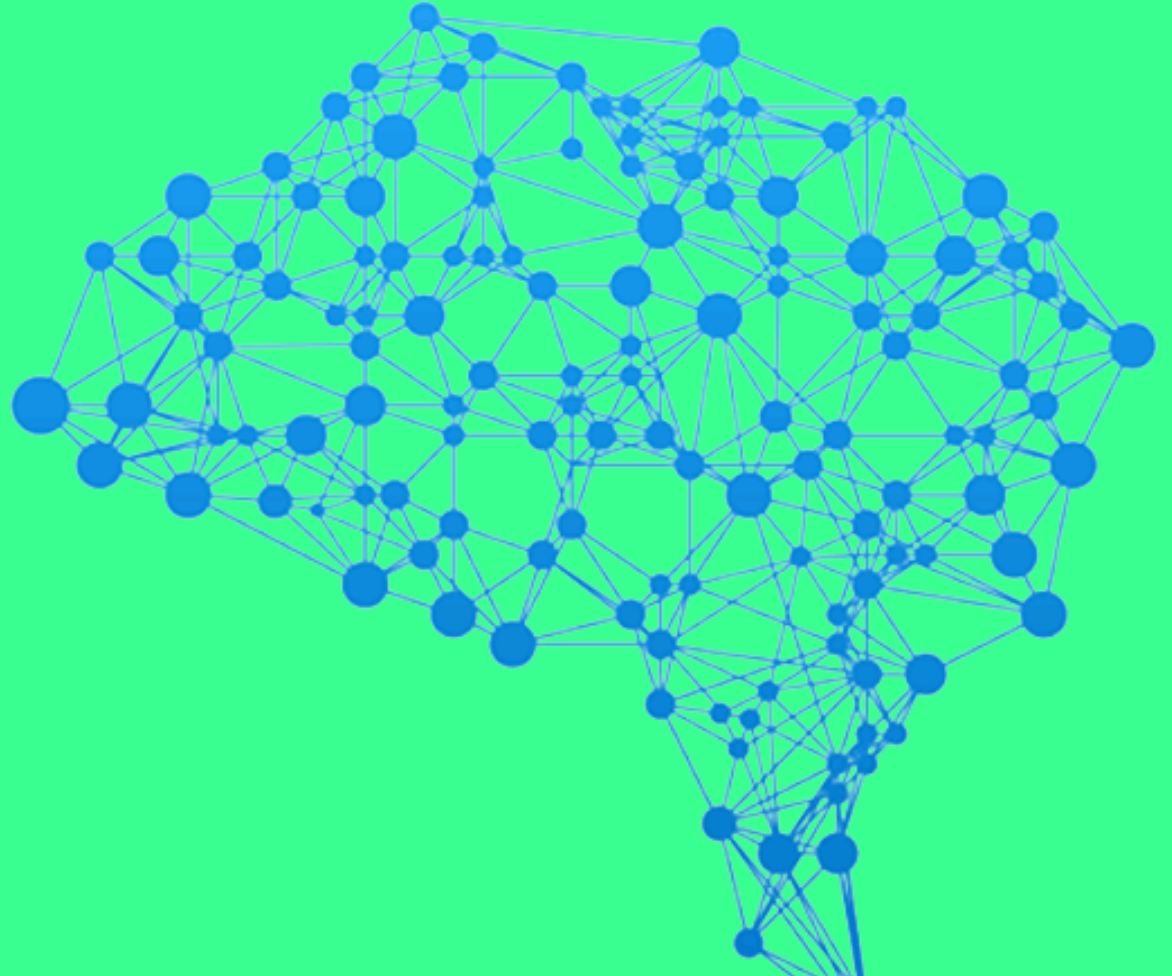
Xarxatec **Activa**

Índice

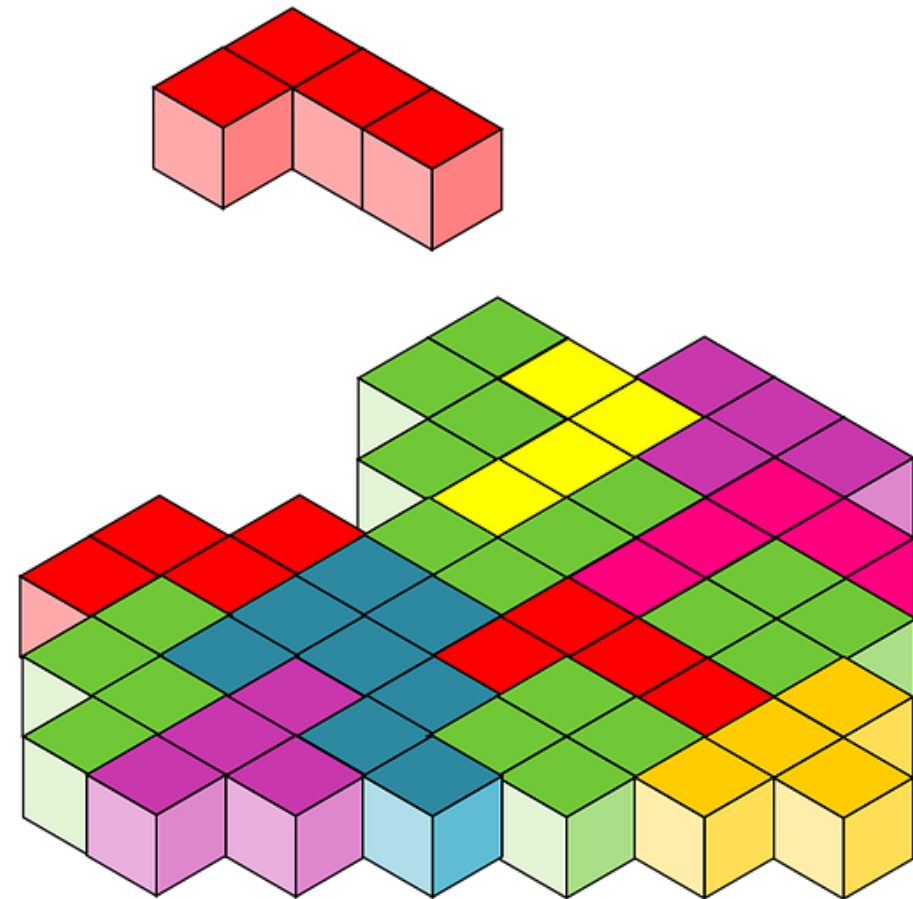
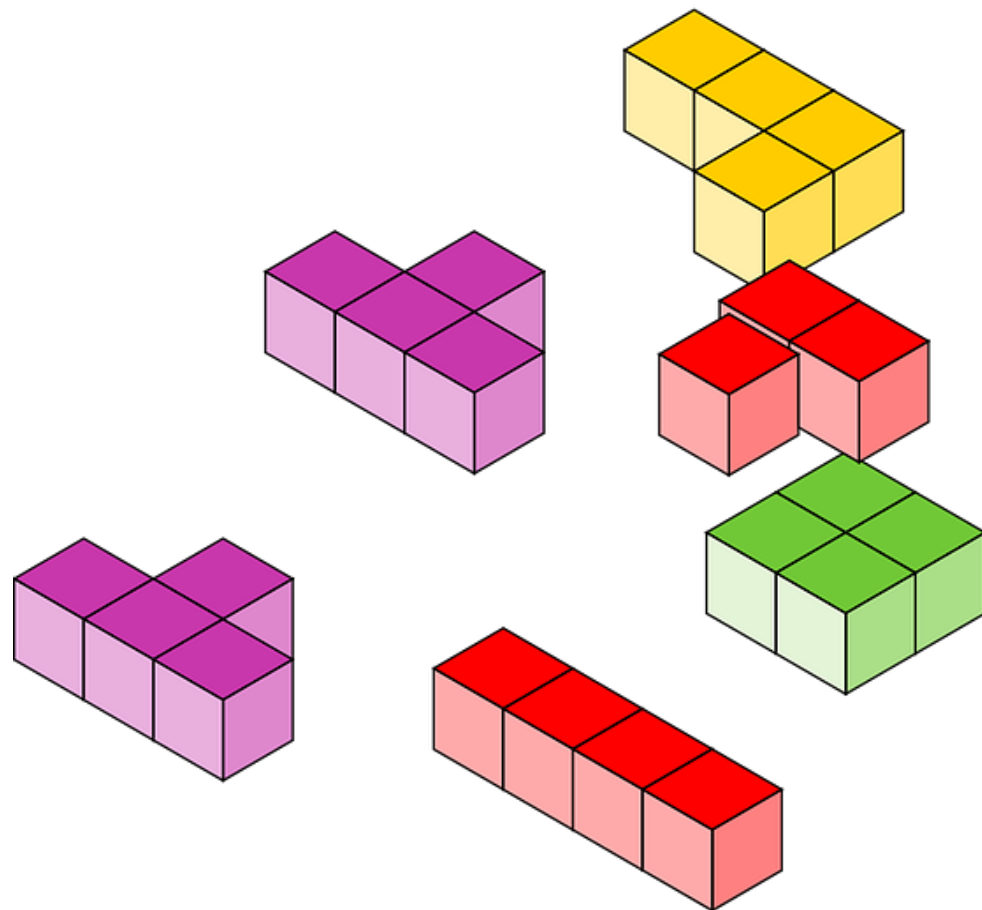
- Pensamiento computacional
- Lenguajes de programación
- Manejar valores: datos simples, operaciones y variables

**Aprendiendo a
programar con
JavaScript**

— Pensamiento computacional



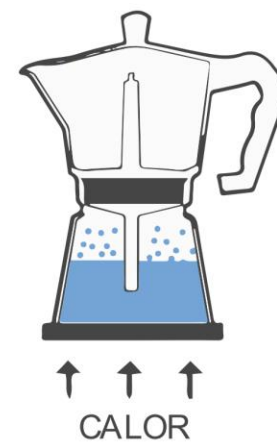
Descomponer los problemas



—
¿Qué es lo que quiero conseguir?



¿Qué necesito para conseguirlo?



El resultado



Diferentes formas de conseguir lo mismo...



Pero conseguir algo simple y funcional suele ser lo óptimo





Algoritmos

Wikipedia: *En matemáticas, lógica, ciencias de la computación y disciplinas relacionadas, un algoritmo (del latín, dixit algorithmus y este del griego arithmos, que significa «número», quizá también con influencia del nombre del matemático persa Al-Juarismi) es un conjunto de instrucciones o reglas definidas y no-ambiguas, ordenadas y finitas que permite, típicamente, solucionar un problema, realizar un cómputo, procesar datos y llevar a cabo otras tareas o actividades.*

En resumen, un algoritmo es cualquier cosa que funcione paso a paso, donde cada paso se pueda describir sin ambigüedad y sin hacer referencia a una herramienta en particular, y además tiene un límite fijo en cuanto a la cantidad de datos que se pueden leer/escribir en un solo paso.



Algoritmos

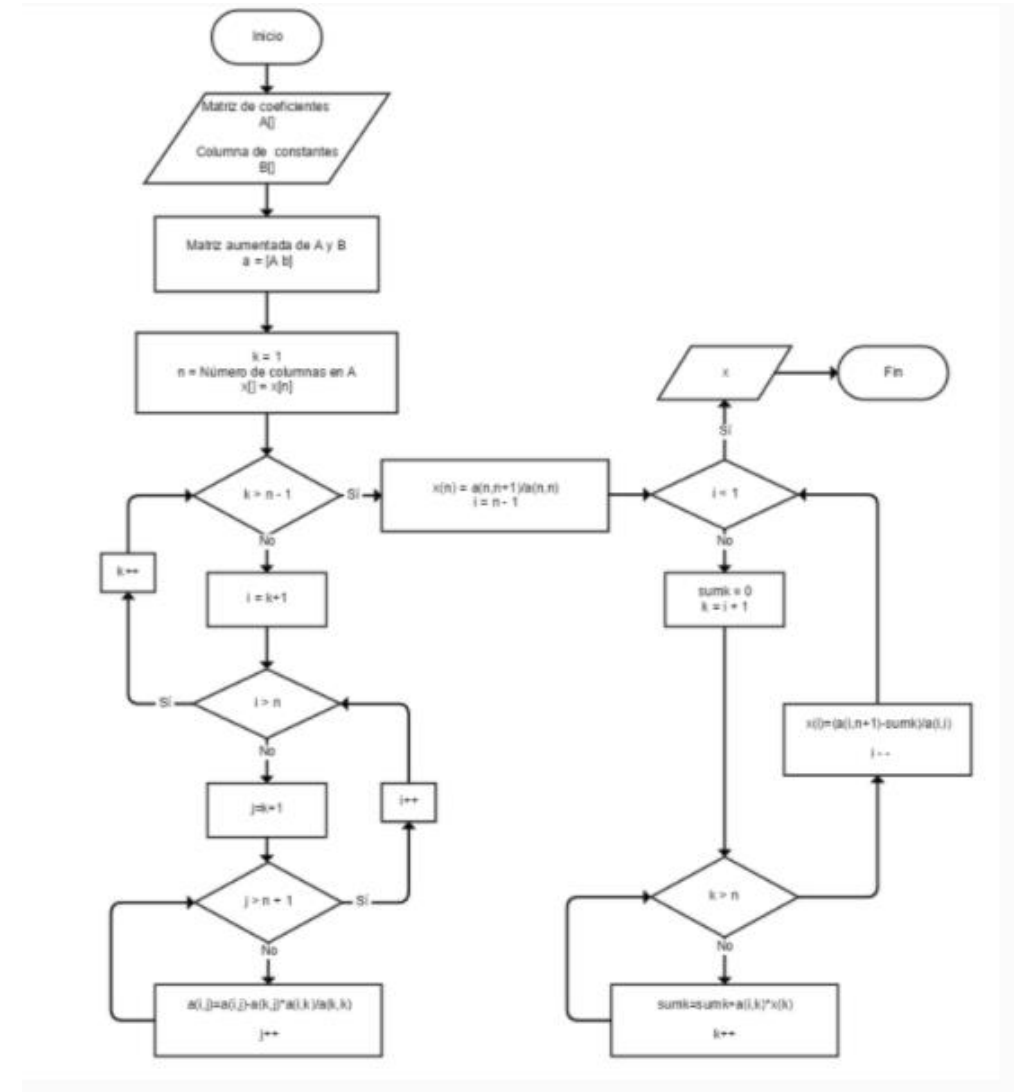
La descripción de un algoritmo usualmente se hace en tres niveles:

- Descripción de alto nivel. Se establece el problema, y se explica el algoritmo de manera verbal, posiblemente con diagramas y omitiendo detalles.
- Descripción formal. Se usa pseudocódigo para describir la secuencia de pasos que encuentran la solución.
- Implementación. Se muestra el algoritmo expresado en un lenguaje de programación específico.

Algoritmos: ejemplo alto nivel

Algoritmo de eliminación de Gauss-Jordan

- Ir a la columna no cero extrema izquierda
- Si el primer renglón tiene un cero en esta columna, intercambiarlo con otro que no lo tenga
- Luego, obtener ceros debajo de este elemento delantero, sumando múltiplos adecuados del renglón superior a los renglones debajo de él
- Cubrir el renglón superior y repetir el proceso anterior con la submatriz restante. Repetir con el resto de los renglones (en este punto la matriz se encuentra en la forma de escalón)
- Comenzando con el último renglón no cero, avanzar hacia arriba: para cada renglón obtener un 1 delantero e introducir ceros arriba de éste sumando múltiplos correspondientes a los renglones correspondientes



Algoritmos: ejemplo formal

```
1. Start
2. Input the Augmented Coefficients Matrix (A):
    For i = 1 to n
        For j = 1 to n+1
            Read Ai,j
        Next j
    Next i
```

```
3. Apply Gauss Jordan Elimination on Matrix A:
    For i = 1 to n
        If Ai,1 = 0
            Print "Mathematical Error!"
            Stop
        End If
        For j = 1 to n
            If i ≠ j
                Ratio = Aj,1/Ai,1
                For k = 1 to n+1
                    Aj,k = Aj,k - Ratio * Ai,k
                Next k
            End If
        Next j
    Next i
```

```
4. Obtaining Solution:
    For i = 1 to n
        Xi = Ai,n+1/Ai,1
    Next i
5. Display Solution:
    For i = 1 to n
        Print Xi
    Next i
6. Stop
```


Algoritmos: ejemplo implementación (en JavaScript)

```
function print(M, msg) {
  console.log("=====" + msg + "=====")
  for(var k=0; k<M.length; ++k) {
    console.log(M[k]);
  }
  console.log("=====")
}

function diagonalize(M) {
  var m = M.length;
  var n = M[0].length;
  for(var k=0; k<Math.min(m,n); ++k) {
    // Find the k-th pivot
    i_max = findPivot(M, k);
    if (A[i_max, k] == 0)
      throw "matrix is singular";
    swap_rows(M, k, i_max);
    // Do for all rows below pivot
    for(var i=k+1; i<m; ++i) {
      // Do for all remaining elements in current row:
      var c = A[i][k] / A[k][k];
      for(var j=k+1; j<n; ++j) {
        A[i][j] = A[i][j] - A[k][j] * c;
      }
      // Fill lower triangular matrix with zeros
      A[i][k] = 0;
    }
  }
}

function findPivot(M, k) {
  var i_max = k;
  for(var i=k+1; i<M.length; ++i) {
    if (Math.abs(M[i][k]) > Math.abs(M[i_max][k])) {
      i_max = i;
    }
  }
  return i_max;
}
```

```
function swap_rows(M, i_max, k) {
  if (i_max != k) {
    var temp = A[i_max];
    A[i_max] = A[k];
    A[k] = temp;
  }
}

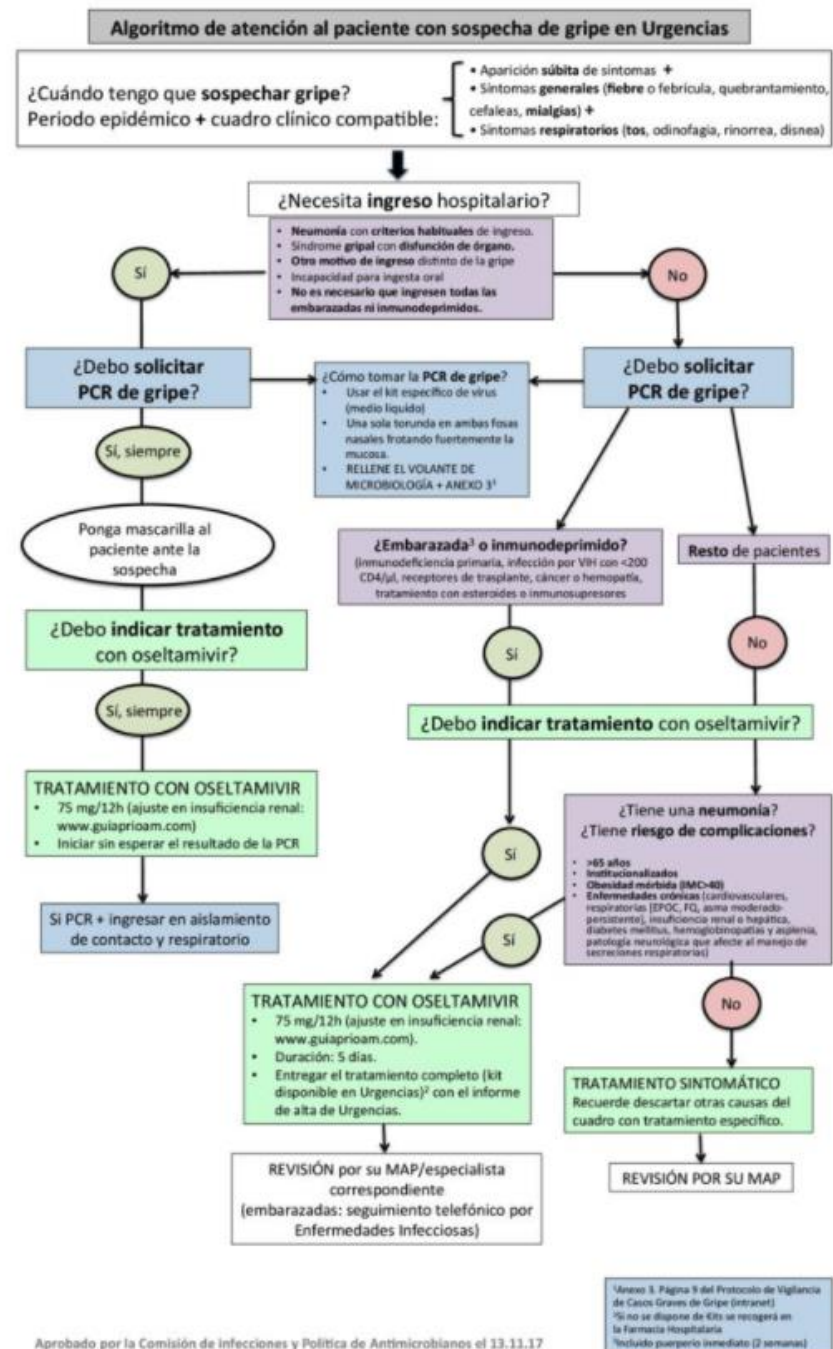
function makeM(A, b) {
  for(var i=0; i<A.length; ++i) {
    A[i].push(b[i]);
  }
}

function substitute(M) {
  var m = M.length;
  for(var i=m-1; i>=0; --i) {
    var x = M[i][m] / M[i][i];
    for(var j=i-1; j>=0; --j) {
      M[j][m] -= x * M[j][i];
      M[j][i] = 0;
    }
    M[i][m] = x;
    M[i][i] = 1;
  }
}
```

```
function extractX(M) {
  var x = [];
  var m = A.length;
  var n = A[0].length;
  for(var i=0; i<m; ++i){
    x.push(A[i][n-1]);
  }
  return x;
}

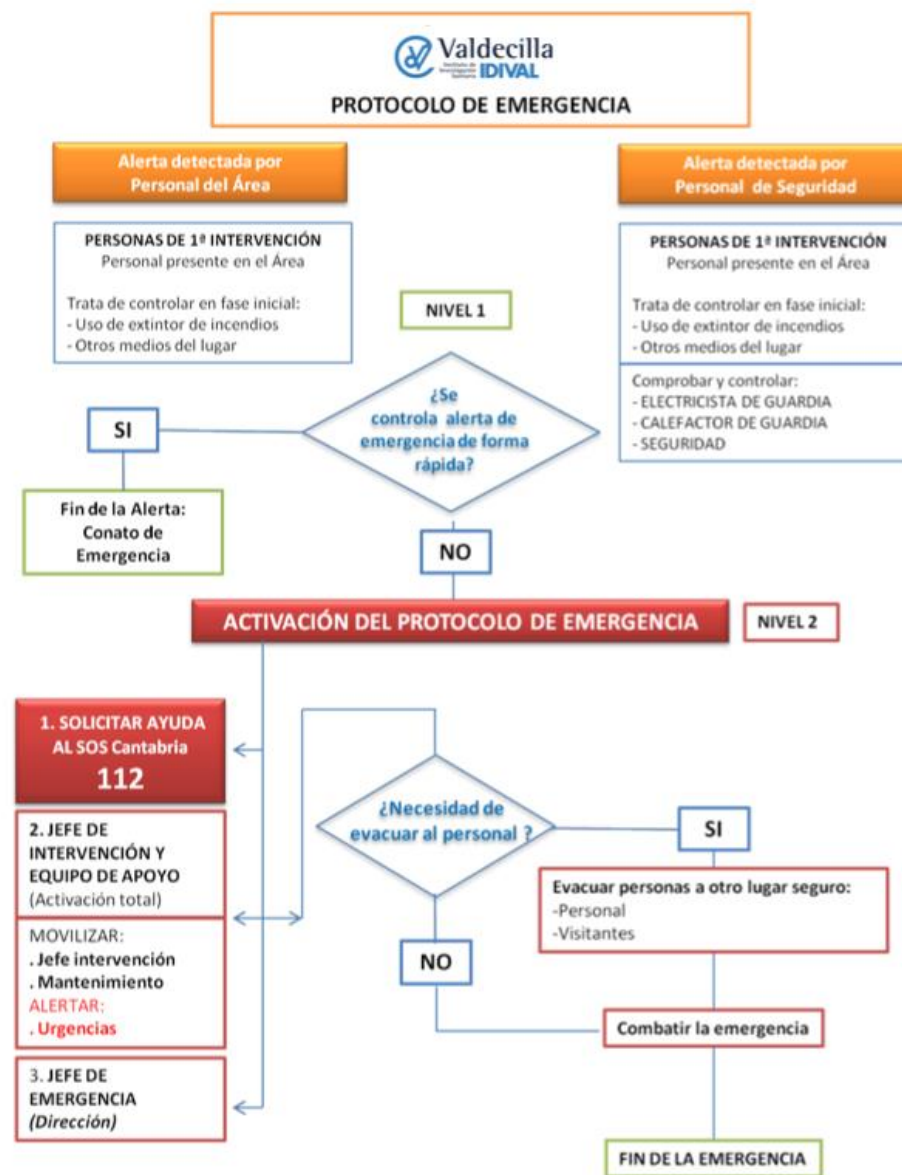
function solve(A, b) {
  //print(A, "A");
  makeM(A,b);
  //print(A, "M");
  diagonalize(A);
  //print(A, "diag");
  substitute(A);
  //print(A, "subst");
  var x = extractX(A);
  //print(x, "x");
  return x;
}
```

Múltiples campos

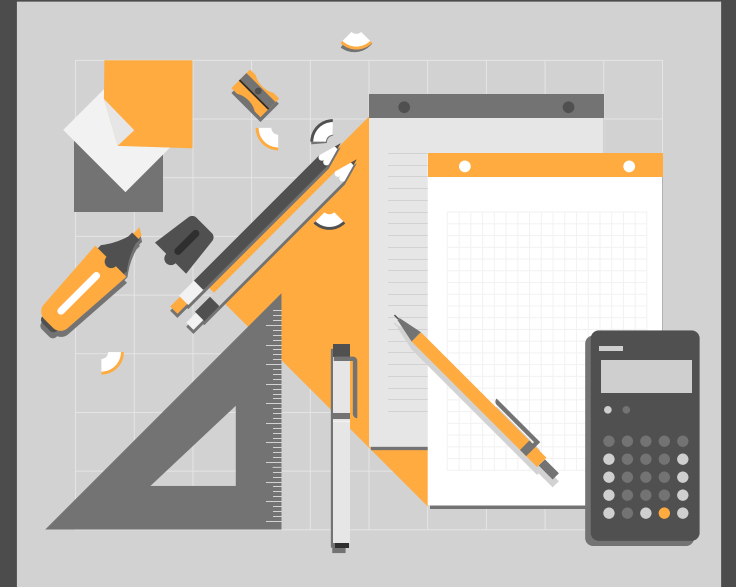


Múltiples campos

ANEXO 1. Algoritmo de actuación en incendios



Seamos más prosaicos...



11/11/2019





Lenguajes de programación

¿Por dónde empezar?



Y de todos esos... ¿por qué JavaScript?

- Indispensable para web dinámica (Frontend)
- Un solo lenguaje para todos los contenidos del curso (Backend con Node.js)
- Dentro de los LP es uno de los fáciles para aprender
- Gran salida laboral

Muchas razones explicadas detenidamente aquí:

<https://www.freecodecamp.org/news/what-programming-language-should-i-learn-first-19a33b0a467d/>



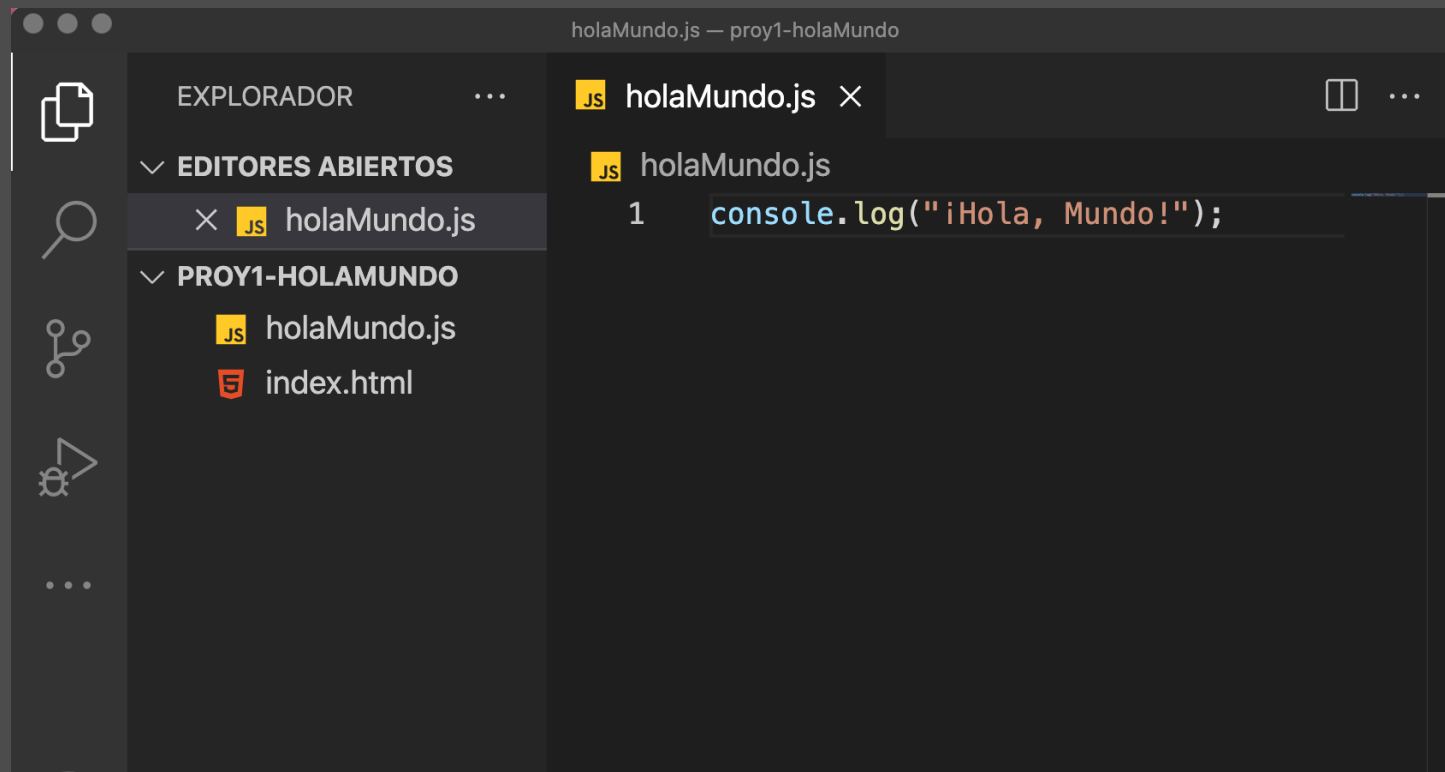
JavaScript

JavaScript (a partir de ahora JS) es un LP de alto nivel intepretado. Se define como multiparadigma aunque eminentemente orientado a objetos basado en propotipos, débil y dinámicamente tipado. El significado de todos estos conceptos, que corresponden a la ontología de los lenguajes de programación, quedará claro a medida que el curso avance.

Se utiliza principalmente del lado del cliente, interpretado por un navegador web, con el fin de mejorar la experiencia del usuario, creando webs dinámicas e interfaces de apps web.

También se utiliza mucho a nivel de servidor para el crear el backend de una app web mediante una extensión del lenguaje llamada Node.js

¡Hola, Mundo!



— Manejar valores: datos simples, operaciones y variables



Un poco más de pseudocódigo...

Elabora un algoritmo para, dada la duración de un curso, calcule el número de sesiones formativas que se necesitarían para completarlo.





Pedir valores a usuario

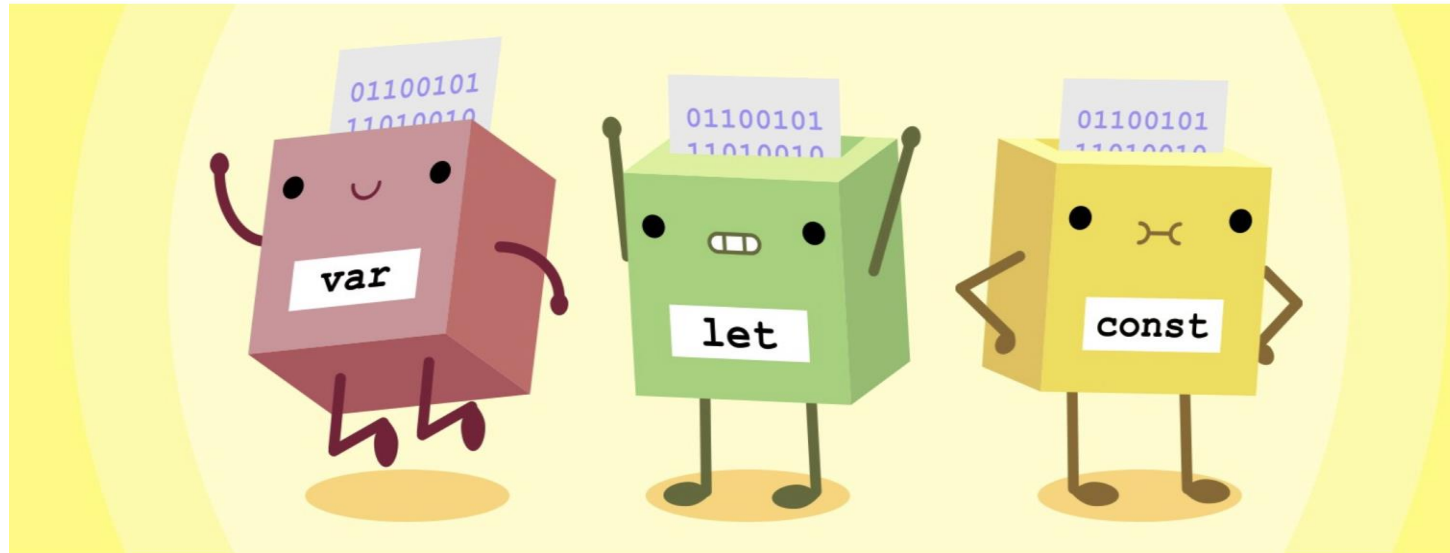
La forma habitual de pedir algún dato a un usuario en una aplicación web es mediante un formulario. Aún no tenemos habilidades adquiridas para tratar correctamente formularios (¡sí para crearlos! Algo es algo...) así que usaremos una función predefinida en JavaScript:

```
prompt("mensaje que mostraremos");
```

Que mostrará una ventana con el mensaje que pasemos a la función y tendrá una celda de texto para que escribamos el dato.

Guardar el dato pedido: variables

Bien, ya sabemos pedir un dato. Pero a menos que tengamos un sistema para guardar ese dato para utilizarlo posteriormente, no servirá para nada. Para ello contamos con las variables.



Declaración y asignación

Los nombres de las variables deben ser muy descriptivos, para facilitar la comprensión del código. Hay que evitar a toda costa utilizar nombres genéricos y desprovistos de significado sin contexto (por ejemplo: a, b, c, fecha, numero, num1, num2, ...)

Aquí asignamos un valor directamente nosotros. ¿Pero podríamos combinarlo con la función prompt?

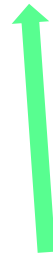
`let duracionCurso = 600;`

Operadores de asignación:
espera un valor al otro lado.

Let nos permite declarar una variable. También const (y var, aunque esta ya no es del todo recomendable). La principal diferencia entre las 3 es pronto para entenderla, pero tiene que ver con el alcance (scope) dentro del código. Es decir, desde dónde, dentro de mi código, puedo alcanzar a ver el valor de la variable y desde dónde no. De momento, y muy importante también, si una vez asignado un valor a una variable, este es posible que cambie, utilizaremos let, si no va a cambiar a lo largo de mi programa, utilizaremos const.

Pues sí, podemos.

```
const duracionCurso = prompt("Introduce la duración del curso");
```

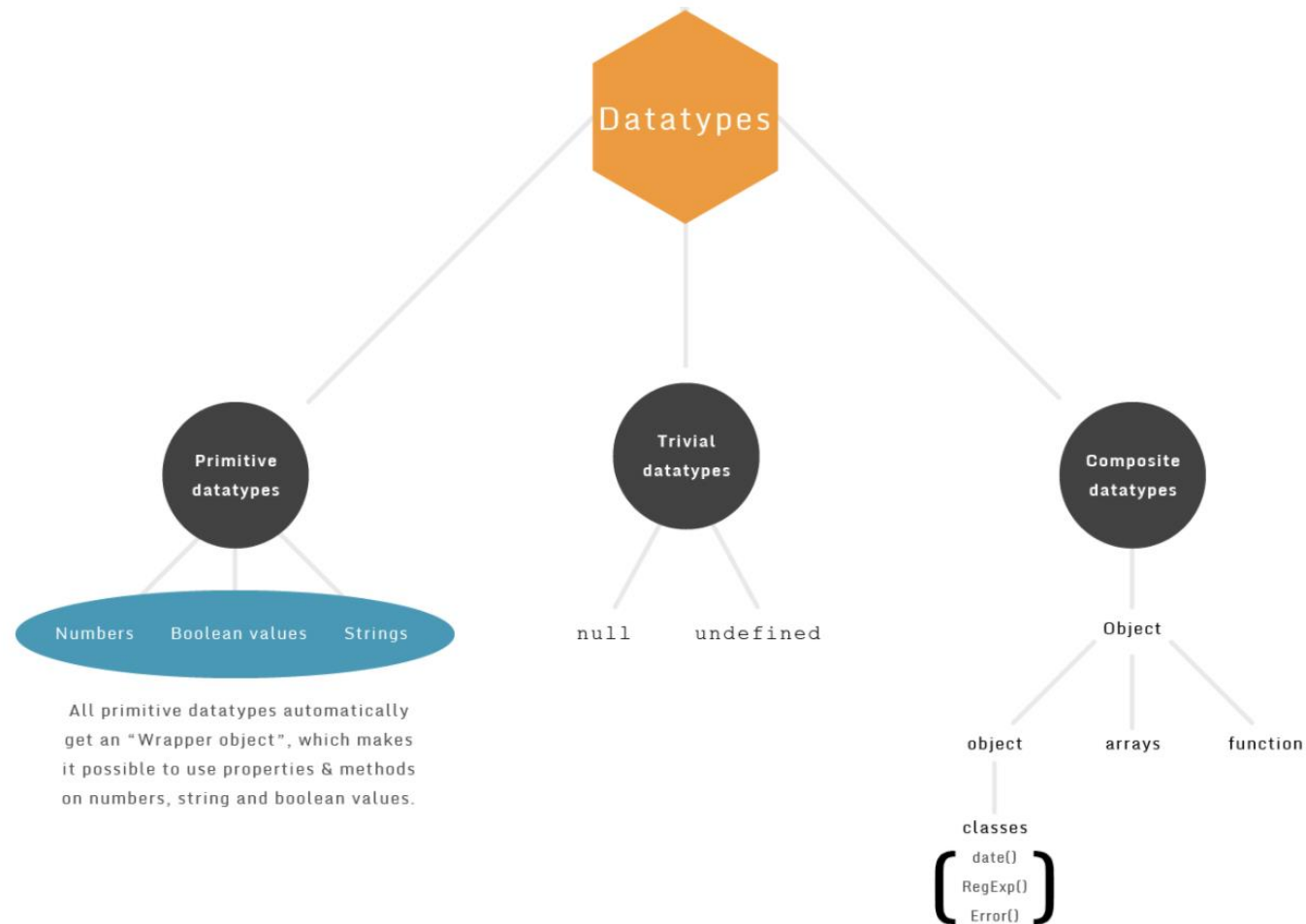


Y esto es debido a que `prompt` es una función. Las funciones las estudiaremos en poco tiempo, pero básicamente son una herramienta para poder reutilizar código de forma sencilla. Las funciones pueden o no tener una respuesta (devolver un valor). En nuestro caso nuestra función `prompt` sí tiene: concretamente devuelve el valor que hayamos escrito en la celda que muestra con el mensaje.

Además, hemos cambiado `let` por `const`, y esto lo hemos hecho porque el valor de `duracionCurso` será siempre el mismo, en todo el programa, nunca cambiará.

¡Bien! Ya puedo guardar un valor. Pero ¿qué puede ser un valor?

JavaScript: lenguaje **débilmente** tipado





Operadores.

Cuando estamos trabajando con números:

1. () Paréntesis.
2. Potencia: **
3. Multiplicación: * -- División: /
4. Resto de la división: %
5. Suma: + -- Resta: -

Perfecto. Ya podemos calcular el número de sesiones ¿No?



Sí podemos.

```
const duracionCurso = prompt("Introduce la duración del curso");  
const duracionSesion = prompt("Introduce la duración de cada sesión");  
const numeroSesiones = duracionCurso/duracionSesion;  
console.log("El número de sesiones es: ", numeroSesiones);
```

Pero...

—
Siempre hay peros.

JavaScript: lenguaje **dinámicamente tipado**



Atención:

¿qué pasa si no trabajamos con números?

O peor aún, ¿qué pasa si no controlamos con lo que estamos trabajando?

Por ejemplo: **prompt**

Solución: parseInt()

```
const duracionCurso = prompt("Introduce la duración del curso");  
const duracionSesion = prompt("Introduce la duración de cada sesión");  
const numeroSesiones = Math.ceil(duracionCurso/duracionSesion);  
console.log("El número de sesiones es: ", numeroSesiones);
```

Y ya tenemos nuestro programa



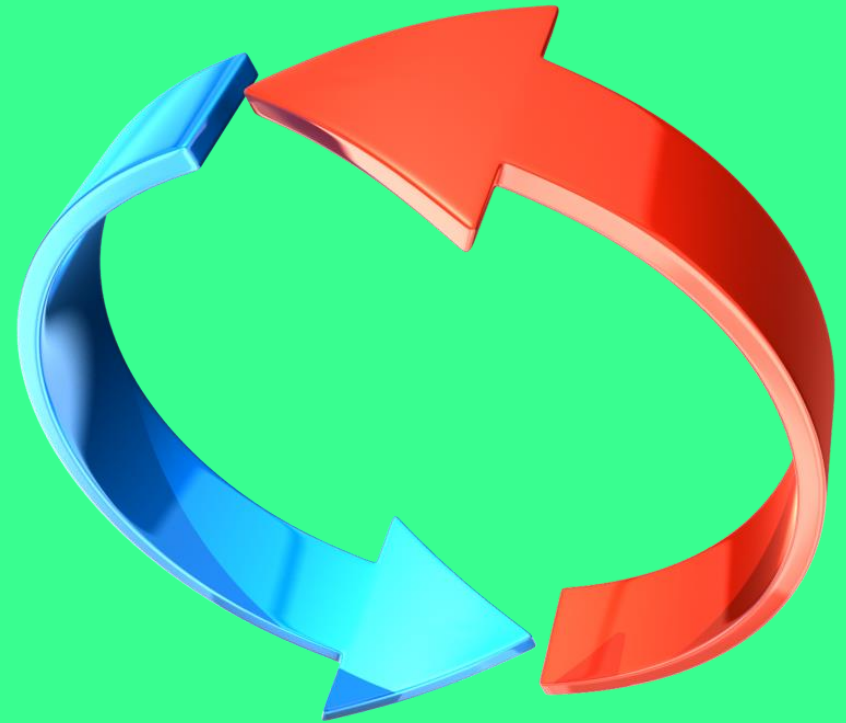
HolaMundo personalizado

Vamos a cambiar el programa HolaMundo, para ello le pedirás al usuario su nombre y mostrarás por consola el mensaje:

"¡Hola *nombreUsuario*!"



— Control de flujo



¿Pares o nones?

¿Podrías pensar en un algoritmo que reflejara el proceso de jugar a pares o nones contra el ordenador?



Generación de números aleatorios

Con la función `Math.random()` generamos un valor aleatorio decimal entre 0 y 1.

Para generar un número entero en un mínimo y un máximo utilizamos una evolución de esa función:

```
Math.floor((Math.random() * (max - min + 1)) + min);
```

`Math.floor()` es una función que devuelve, a partir de un número decimal, el número entero redondeado de la siguiente manera:

`Math.floor(6.8)` devolverá 6

`Math.floor(-6.8)` devolverá -7



Operadores lógicos y tipo booleano

Un valor booleano se evalúa a TRUE (verdadero, 1) Y FALSE (falso, 0)

Cualquier condición con un operador lógico se valorará a True o False.

Compruébalo: `console.log(3>5);`

Los operadores de comparación son: `>`, `>=`, `<`, `<=`, `==`, `!=`, `===`, `!==`

y los operadores lógicos: `&&`, `||`, `!`



Control de flujo

Como hemos comentado un programa es una ejecución secuencial de instrucciones que manipulan datos para conseguir un resultado deseado.

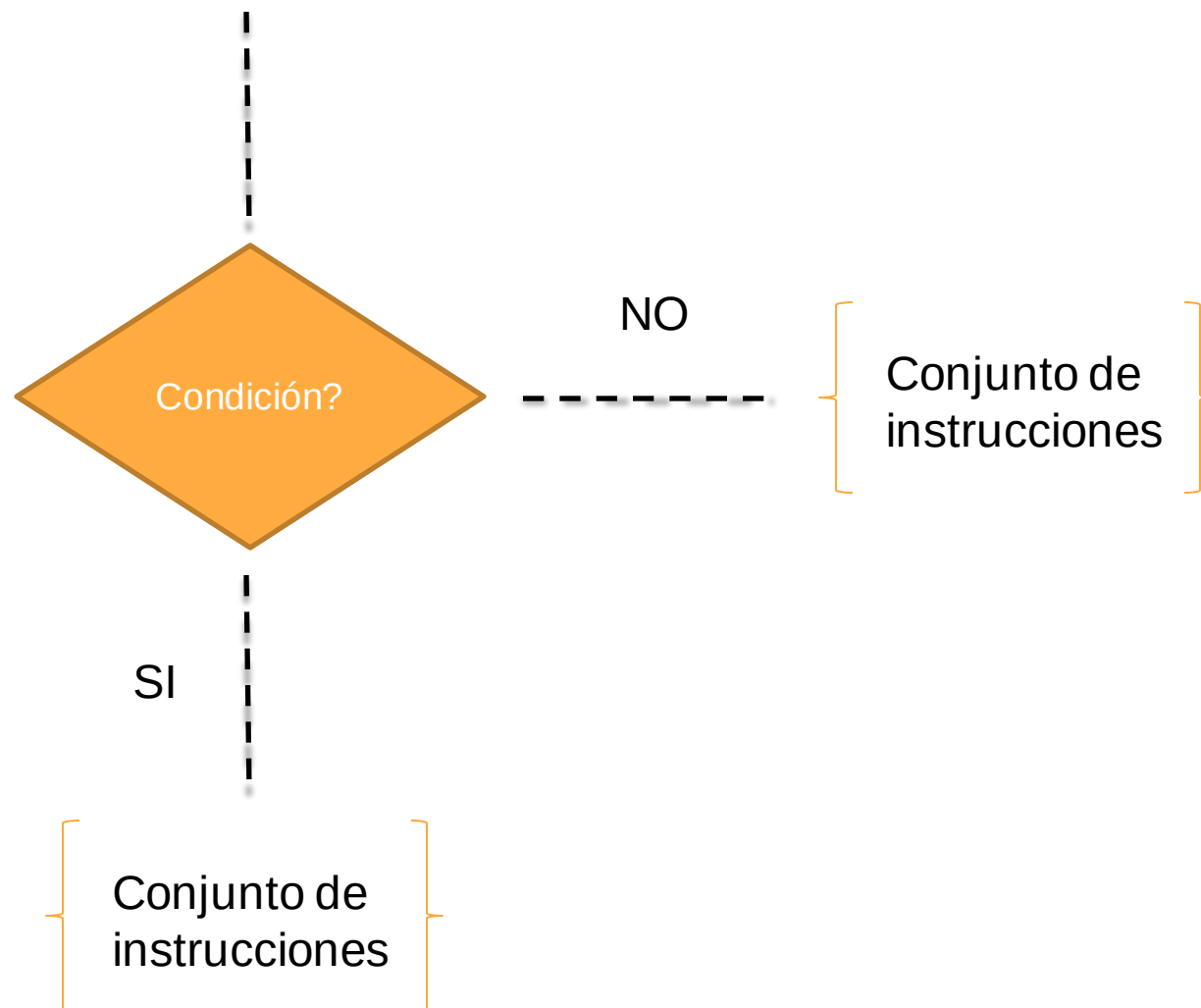
En muchas situaciones vamos a necesitar romper ese flujo secuencial de instrucciones, es decir; vamos a querer controlar el flujo del programa.

Una primera situación, de todas las que vamos a estudiar es el caso en el que queremos bifurcar el código. Esto se produce cuando queremos valorar si una situación determinada se cumple o no. Si se cumple haríamos una cosa y si no otra (o si se cumple haríamos una cosa y si no no la haríamos).

Por ejemplo: ¿tienes azúcar? Si lo tienes continúa con el programa, si no, ve a comprar.

Otro ejemplo: ¿el valor que me has pasado es un número? Si lo es continúa con el programa, si no acaba el programa (o pídelo de nuevo, aunque esto es más peliagudo, como veremos más adelante XD)

Control de flujo



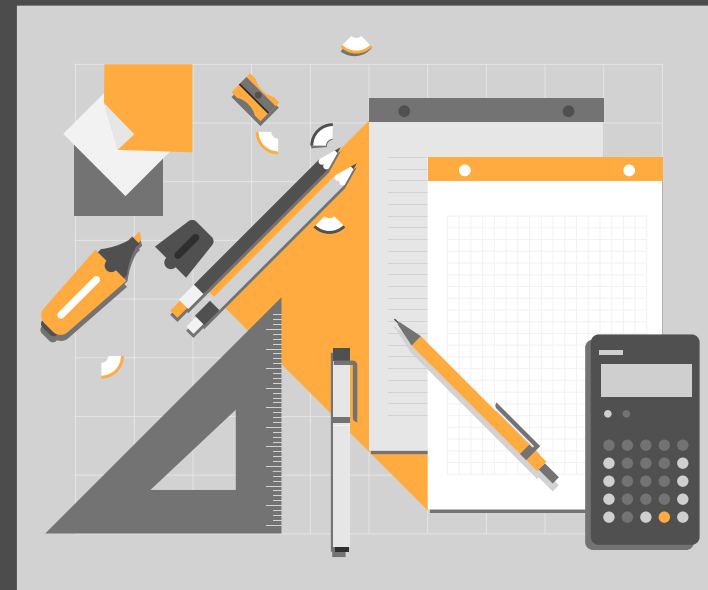
Toma de decisiones: estructura de control if-else

Para poder hacer esas bifurcaciones utilizamos las sentencias if-else:

```
...  
If (condicion){  
instrucciones que se ejecutarán si se cumple la condición;  
} else {  
instrucciones que se ejecutarán si NO se cumple la condición;  
}  
...
```

¿Piedra, papel o tijera?

Primero piensa en un algoritmo que te permita jugar con el ordenador a piedra, papel o tijera.
Después impleméntalo.



Evaluar varias condiciones: AND y OR

AND (&&) se evalúa como TRUE siempre y cuando todas las condiciones encadenadas son ciertas.

OR (| |) se evalúa como TRUE cuando al menos una de las condiciones encadenadas es cierta.

Ejemplo:

```
const sexo = "mujer";  
const edad = 54;
```

```
If (sexo = "mujer" && edad>=40) : TRUE  
If (sexo = "hombre" && edad>=40) : FALSE  
If (sexo = "mujer" || edad>=40) : TRUE  
If (sexo = "mujer" || edad<40) : TRUE  
If (sexo = "hombre" || edad>40) : TRUE  
If (sexo = "hombre" || edad<40) : FALSE
```




Error en el prompt

Estamos asumiendo tanto en el ejercicio de pares o nones como en el de piedra, papel o tijera que la elección del usuario será correcta: que escogerá exactamente una de las opciones escribiéndola correctamente. Pero muchas veces eso puede no ocurrir, así que necesitamos un mecanismo para poder repetir decisiones, o parte del código del programa.

En el caso de querer controlar el flujo para repetir una serie de instrucciones, existen dos alternativas principales:

- Sabes de antemano cuántas veces quieres repetir el código.
- No lo sabes.



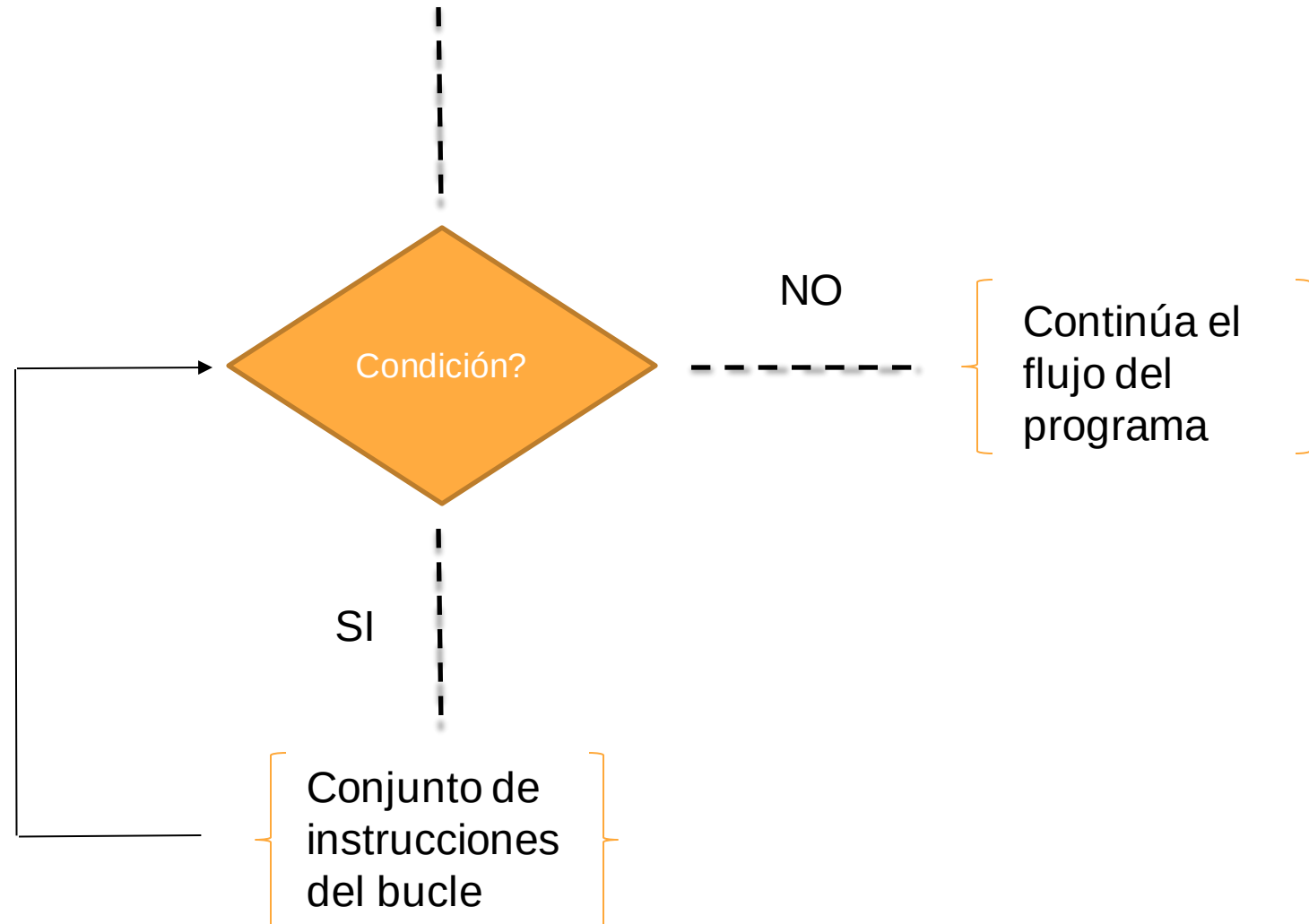
Repetir indefinidas veces: WHILE

Cuando queremos repetir un número indeterminado de veces un número determinado de instrucciones utilizamos el bucle WHILE.

Ejemplos: si un usuario no escoge bien pares o nones, se lo volvemos a pedir ¿Pero sabemos cuántas veces se va a equivocar?

Ejemplo más real: queremos logearnos en una web. Si escribimos mal el usuario o la contraseña volveremos a mostrar el formulario de inscripción. ¿Sabemos cuántas veces se puede equivocar el usuario?

Control de flujo: bucle WHILE



Control de flujo: bucle WHILE

...

```
while (condicion){  
instrucciones que se ejecutarán si se cumple la condición;  
}
```

...

Control de flujo: bucle DO-WHILE

Si las instrucciones del bucle while se ejecutan SIEMPRE al menos una vez, utilizamos la forma do-while:

...

```
do {  
  instrucciones que se ejecutarán si se cumple la condición;  
} while (condicion)
```

...

Controlar que se pide la opción correcta

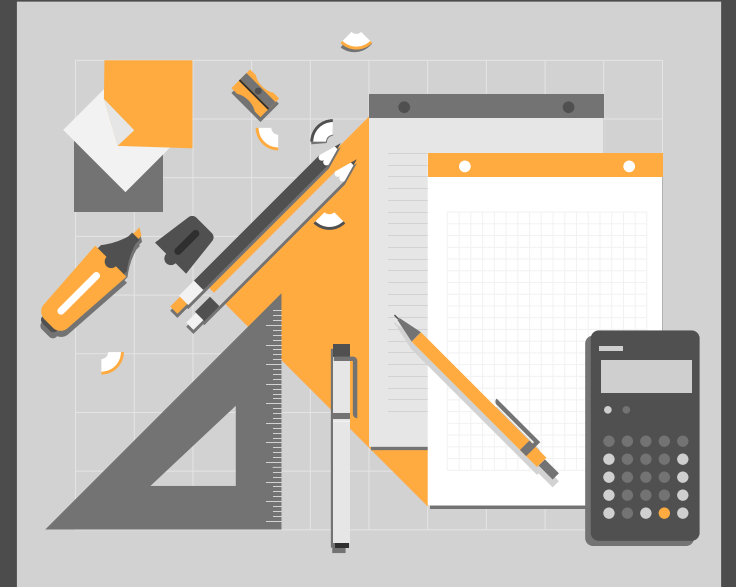
...

```
let opcionUsuario = prompt("Escoge pares o nones");  
while (opcionUsuario!="pares" && opcionUsuario!="nones"){  
    opcionUsuario = prompt("La elección no es válida. Escoge pares o  
nones");  
}
```

...

Page 10 of 10

Luego intenta cambiar el programa para que se juegue varias veces: concretamente hasta que el usuario o la Máquina lleguen a 3 puntos.



Incrementar una variable

Existen varias formas de incrementar una variable de tipo numérica un número determinado de unidades:

Por ejemplo, tenemos creada la variable `numeroBase` y queremos incrementarla 3 unidades, podríamos hacerlo de varias formas:

- `numeroBase = numeroBase + 3;`
- `numeroBase += 3;`

Si el incremento es de una unidad, además de las dos anteriores tenemos otra más reducida:

- `numeroBase++;`

Para reducir una unidad tendríamos también las 3 formas:

- `numeroBase = numeroBase - 1;`
- `numeroBase -= 1;`
- `numeroBase--;`



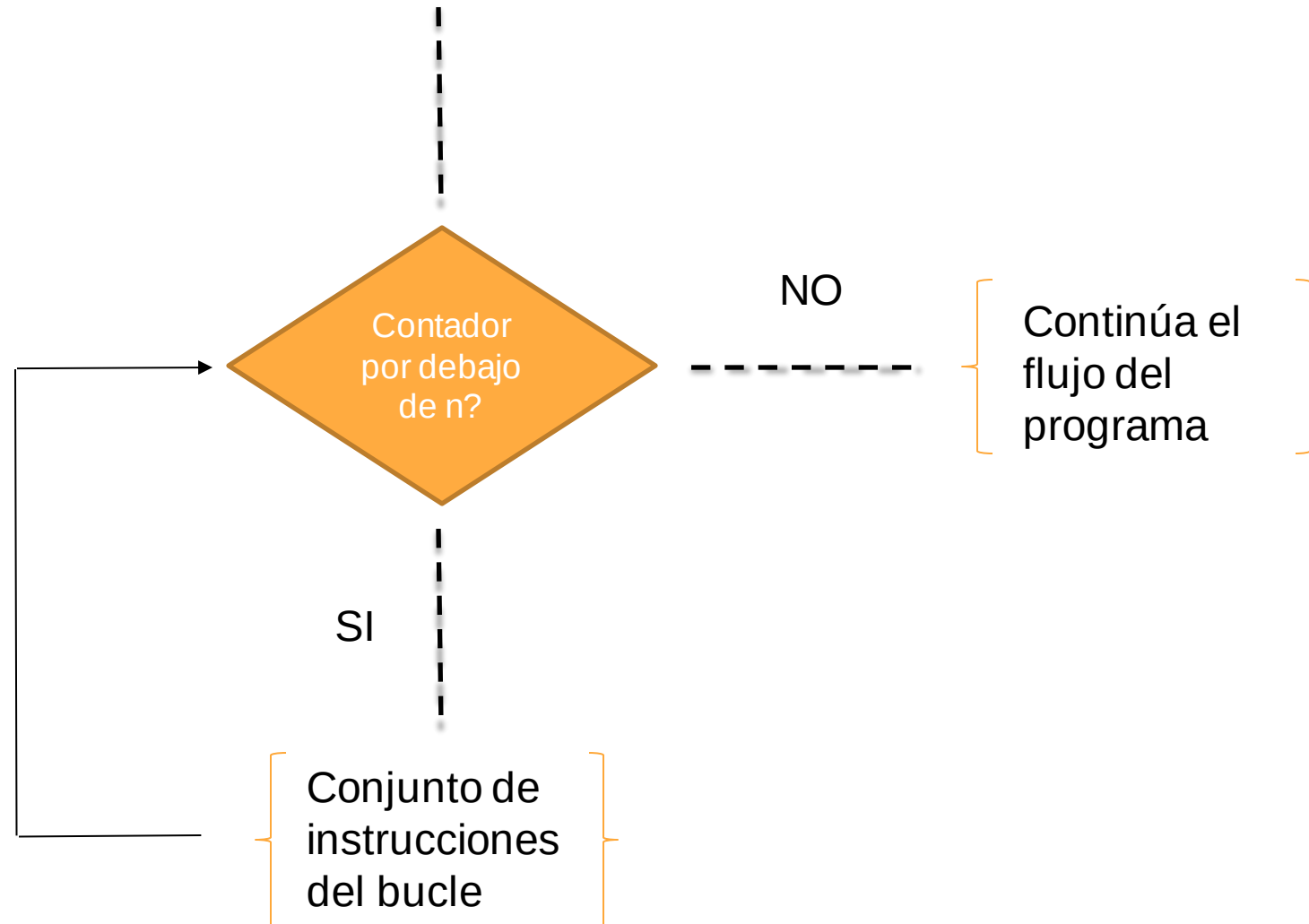
Repetir n veces: Bucle FOR

A diferencia de lo que ocurre cuando utilizamos el bucle WHILE, es posible que sepamos de antemano cuántas veces quiero que se ejecuten las instrucciones que se repiten, para ello tenemos el bucle FOR.

Para controlar un bucle FOR necesitamos una variable creada exproceso solo para ello, a la que se conoce normalmente como contador (y que se suele nombrar como i, aunque esto es un convenio que no es obligatorio utilizar, claro está).

Cada vez que el bucle haga una iteración completa, incrementaremos ese contador y cuando llegue al número de repeticiones que tiene que hacer acabaremos la ejecución del bucle y continuaremos el flujo normal del programa.

Control de flujo: Bucle FOR para n repeticiones



Control de flujo: Bucle FOR para n repeticiones

...

```
for(let i=1; i<=n; i++){  
    instrucciones que se ejecutarán n veces;  
}
```

...

Practicamos el FOR

Crea un programa que muestre por pantalla los 100 primeros números, con la particularidad de que cuando muestre algún número que sea múltiplo de 3 añada la coletilla "múltiplo de 3"





Elección sobre múltiples valores: SWITCH

La estructura if-else permite evaluar una condición a verdadero o falso, es decir, 2 posibilidades, si quiero analizar más posibilidades la opción es encadenar else if para cada opción que quiero evaluar, quedando un código, que, habitualmente, será poco comprensible. Existe otra posibilidad para evaluar (y reaccionar en consecuencia) los diferentes valores de una variable: SWITCH.

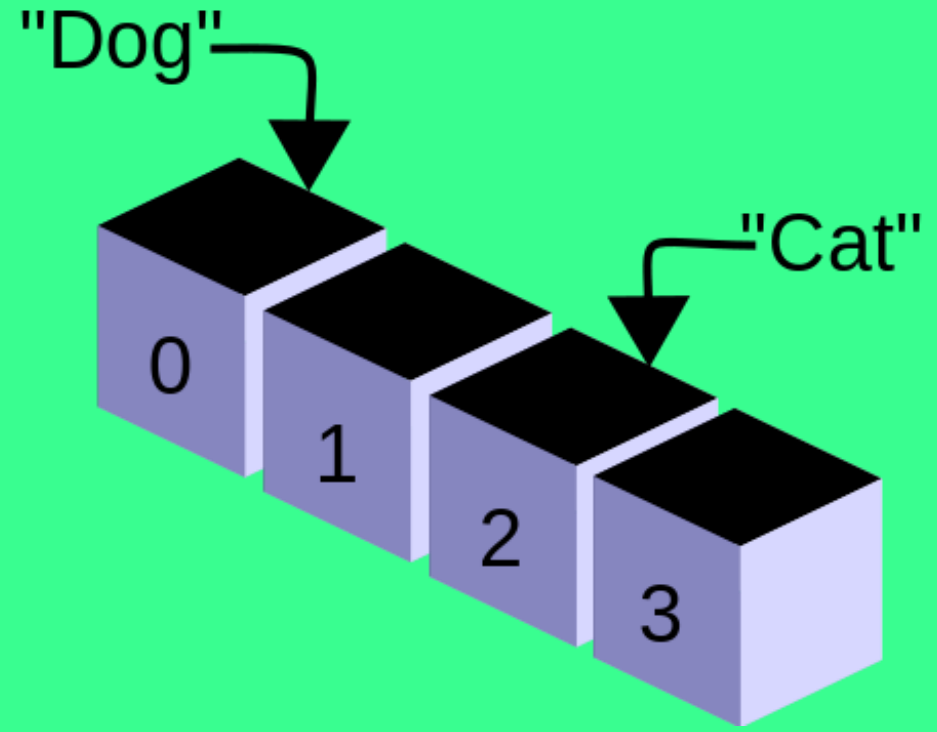
Control de flujo: SWITCH

...

```
switch(variable){  
    Case valor1:  
        Instrucciones que se ejecutan en el caso de que la variable tenga como valor valor1;  
        break;  
    Case valor2:  
        Instrucciones que se ejecutan en el caso de que la variable tenga como valor valor2;  
        break;  
    ...  
    Case valorn:  
        Instrucciones que se ejecutan en el caso de que la variable tenga como valor valorn;  
        break;  
    default:  
        Instrucciones que se ejecutan en caso de que la variable no tenga como valor ninguno de  
los valores evaluados en los cases;  
}
```

...

— Estructuras de datos: array





¿Qué es un array?

Los *arrays* son objetos similares a una lista, que disponen de métodos para efectuar operaciones de recorrido y de mutación. Tanto la longitud como el tipo de los elementos de un *array* son variables.

La longitud de un *array* puede cambiar en cualquier momento, y los datos se pueden almacenar en ubicaciones no contiguas, es decir, pueden existir posiciones vacías (no definidas).

Creación de un array

```
let arrayVacio = [ ];  
//crea un array vacio
```

```
let progLang = ["JavaScript", "C#", "Java", "Python"];  
//crea e inicializa el array con 4 strings
```

```
let mixDataTypesArray = [3, "HolaMundo", 3*"HolaMundo",, 77];  
/*crea e inicializa el array con varios tipos de valores y una posición vacía  
(undefined)*/
```

Acceder a un elemento de un array

```
let arrayEjemplo = ["hola", "qué", "tal", "Xarxatec?"];

let aux = arrayEjemplo[2];

console.log(aux);
/*devolverá el valor de la tercera posición del vector, que en este caso sería
"tal"*/
```

Tamaño de un vector

La propiedad `length` de todos los vectores nos devuelve su tamaño.

```
let progLang = ["JavaScript", "C#", "Java", "Python"];  
console.log(progLang.length); //devolvería 4
```

Esto nos permite controlar la condición de cierre cuando recorro un vector, como veremos un poco más adelante, o por ejemplo acceder a la última posición de un vector, sin tener que comprobar qué índice tiene:

```
let ultimoLenguaje = progLang[progLang.length-1];
```

O el penúltimo:

```
let penultimoLenguaje = progLang[progLang.length-2];
```

Y así sucesivamente.

Añadir elementos a un array

```
let arrayEjemplo = ["hola", "que", "tal", "Xarxatec"];
```

```
arrayEjemplo.push("Activa");
```

```
//añadiría una posición al final del vector con el valor "Activa"
```

```
arrayEjemplo.unshift("Ey!");
```

```
//añadiría una posición al inicio del vector con el valor "Ey!"
```

Eliminar elementos a un array

```
let arrayEjemplo = ["hola", "que", "tal", "Xarxatec"];
```

```
arrayEjemplo.pop();  
//eliminaría la última posición
```

```
arrayEjemplo.shift();  
//eliminaría la primera posición
```

```
ArrayEjemplo.slice(1,3);  
/*a partir de la posición que marca el primer parámetro, elimina tantas  
posiciones como el valor del segundo parámetro. En este caso eliminaría 3  
elementos a partir del segundo elemento, con lo que partiendo del vector  
original quedaría ["hola"]*/
```

Buscar valores en un array

```
let arrayEjemplo = ["hola", "que", "tal", "Xarxatec"];  
  
arrayEjemplo.indexOf("que");  
//devolverá un 1
```

Cambiar elementos de un array

```
let arrayEjemplo = ["hola", "que", "tal", "Xarxatec"];  
arrayEjemplo.slice(1,3, "qué", "tal", "Xarxatec?");
```

/*a partir de la posición que marca el primer parámetro, elimina tantas posiciones como el valor del segundo parámetro. En este caso eliminaría 3 elementos a partir del segundo elemento, con lo que partiendo del vector original quedaría ["hola"] Si además añadimos n valores como parametros de la función, esos valores se añaden uno a uno como posiciones del vector a partir de la posición del primer parámetro. Es decir, en el ejemplo, al final el vector quedaría ["hola", "qué", "tal", "Xarxatec?"]*/

Recorrer un array

Una forma de recorrer un array es utilizando un bucle for:

```
let arrayEjemplo = ["hola", "que", "tal", "Xarxatec"];

for(let i=0; i<arrayEjemplo.length; i++){
    ...
}
```

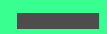
Otra forma habitual es utilizando la función `forEach`:

```
arrayEjemplo.forEach(function(elemento, indice, array){
    ...
});
```


Practicamos con arrays

Dado un vector de números, devuelve por consola el valor máximo, el mínimo y la media de todos los valores almacenados en el vector.





Funciones



¿Qué es una función?

Las funciones tienen el objetivo primordial de reaprovechar el código, empaquetándolo y permitiendo su utilización en diferentes partes de nuestro proyecto, o en otros proyectos.

Una función puede admitir unos datos de entrada (parámetros) y puede devolver algún valor, pero ambas características son opcionales.

Una función puede, además de devolver un valor, modificar información ya existente o hacer ambas cosas.

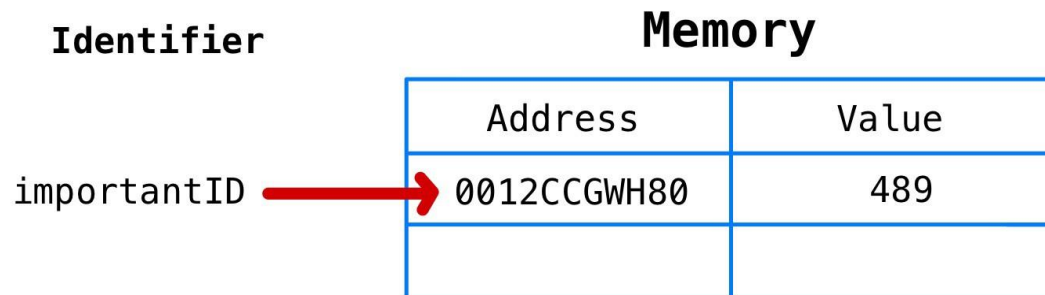
Su estructura es:

```
function nombre(param1, param2,...){  
  ...  
  (return valor) //opcional  
}
```

Funciones: parámetros

Los parámetros son los datos (valores) que pasamos a la función y que se utilizan dentro de esta. Estos datos pueden ser de cualquier tipo simple (number, string, etc) o de cualquier otra estructura (cualquier tipo de objeto, array, diccionario, etc) El paso de parámetros puede ser de dos maneras: Por valor o por referencia.

Para entender este concepto debemos comprender que cuando en un lenguaje de programación creamos una variable (un identificador), lo que hacemos es guardar un puntero a una dirección de memoria (un código que direcciona a una posición de la memoria del ordenador). Es en esa posición de memoria donde se encuentra el valor, el dato en si, almacenado.





Funciones: paso por valor

Cuando en una función un parámetro se pasa por valor, creamos una copia del valor almacenado en el identificador que pasamos como parámetro. Es decir, cualquier cambio que hagamos en la función sobre ese identificador no repercutirá en ninguna variable externa a la función, puesto que no estaríamos trabajando con ellas, habríamos creado una copia del valor almacenado.

En Javascript, cualquier parámetro que guarde un dato de tipo simple (number, string, boolean, null y undefined) se pasa por valor.



Funciones: paso por referencia

Cuando en una función un parámetro se pasa por referencia, lo que estamos haciendo es pasar el puntero a la dirección de memoria que almacena ese valor. Es decir, cualquier cambio que hagamos en la función sobre ese identificador perdurará y afectará a ese identificador concreto y el valor al que apunta, fuera del ámbito de la propia función.

En Javascript, cualquier parámetro que guarde un dato que no sea de tipo simple se pasa por referencia.

Practicamos funciones

Modifica el anterior ejercicio para que el proceso de calcular el máximo, el mínimo y la media del vector sea, cada uno, una función definida e implementada por ti y utilízalas sobre el mismo vector que utilizaras como ejemplo en el anterior ejercicio.

